

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 7 JSON y MongoDB

Alumno: Tomás Rodeghiero
Año: 2026

Resolución basada en el Teórico 9 (XML y JSON) y en experimentación con JsonPath/Gson en Java,
MongoDB y Compass.

Índice

1. Introducción	2
2. Ejercicio 1: Consultas JsonPath sobre el inventario	2
3. Ejercicio 2: Generar un JSON único de facturas e items	5
4. Ejercicio 3: Diseño de la base en MongoDB	7
5. Ejercicio 4: Carga mínima	10
6. Ejercicio 5: Consultas MongoDB	12
Conclusiones	14

1. Introducción

El Práctico 7 introduce dos tecnologías centrales del modelo documental: **JSON** como formato semiestructurado y **MongoDB** como motor de base de datos orientado a documentos. Los cinco ejercicios cubren ambos lados:

- **Ejercicios 1 y 2:** trabajo con JSON desde Java. Consultas a un documento JSON con JsonPath (estilo XPath para JSON) y generación programática de un documento JSON a partir de una base relacional con la biblioteca Gson.
- **Ejercicios 3 a 5:** modelado en MongoDB. Diseño de colecciones, carga de datos y consultas (incluyendo \$lookup para joins entre colecciones).

Cada ejercicio sigue siempre la misma estructura: **Consigna** (texto literal del PDF), **Idea** (lectura natural y concepto detrás), **Solución** (JsonPath, Java, mongosh según corresponda), **Por qué funciona** (lógica paso a paso) y **En MySQL 8+ (JSON)** (equivalente con columnas JSON y funciones nativas: JSON_EXTRACT, JSON_TABLE, JSON_OBJECT, JSON_ARRAYAGG, operadores -> y ->>).

Recursos del práctico. La cátedra provee un proyecto Java con Gson y JsonPath, copiado en `practico-07/recursos/json/`. Los documentos de ejemplo son `json.txt` (de la teoría) y `jsonInventario.txt` (del Ejercicio 1). Para MongoDB se asume mongosh 1.x o superior conectando a un servidor 6.x+.

2. Ejercicio 1: Consultas JsonPath sobre el inventario

Consigna

Dado el documento `jsonInventario.txt` realice las siguientes consultas:

- Todos los items del inventario.
- Nombres de todos items del inventario.
- Item que están bajos en stock.
- Colores de los items cuyo alto es más de 10 cm.

Idea

JsonPath es un lenguaje de consulta para documentos JSON inspirado en XPath: \$ representa la raíz, . navega hijos, [*] es comodín de arreglo y [?(...)] es un predicado de filtro. El concepto detrás del ejercicio es que un documento JSON anidado se puede explorar y filtrar declarativamente, igual que XPath sobre XML, sin necesidad de recorrer el árbol con código imperativo.

Solución

Estructura del documento:

```
{
  "inventario": {
    "items": [
      { "nombre": "tv",          "cantidad": 25, "status": "A",
        "tamano": { "alto": 14,   "ancho": 21,   "unidad": "cm" },
        "colores": ["negro", "amarillo"] },
      { "nombre": "notebook", "cantidad": 50,  "status": "A",
        "tamano": { "alto": 8.5,  "ancho": 11,   "unidad": "pulgadas" },
```

```

    "colores": ["rojo","negro"] },
    { "nombre": "mesa",      "cantidad": 100, "status": "D", ... },
    { "nombre": "planner",   "cantidad": 75,  "status": "D", ... },
    { "nombre": "postcard",  "cantidad": 45,  "status": "A",
      "tamanio": { "alto": 10,    "ancho": 15.25, "unidad": "cm" },
      "colores": ["azul"] }
  ]
},
"stock_min": 60
}

```

Observaciones útiles: `stock_min` está en la raíz (no dentro de cada ítem); la unidad varía entre cm y pulgadas (hay que filtrar por unidad para comparar alturas).

a) Todos los ítems:

```
$.inventario.items[*]
```

b) Nombres de todos los ítems:

```
$.inventario.items[*].nombre
-- Resultado: ["tv","notebook","mesa","planner","postcard"]
```

c) Ítems con bajo stock (cantidad < stock_min):

```
$.inventario.items[?(@.cantidad < $('stock_min'))]
-- Cumplen: tv (25), notebook (50), postcard (45)
```

d) Colores de ítems con alto > 10 cm (filtrar por unidad y alto):

```
$.inventario.items[?(@.tamanio.unidad == 'cm' && @.tamanio.alto > 10)].colores[*]
-- Solo tv (alto=14, cm) cumple; postcard tiene alto=10 (estricto, no entra)
-- Resultado: ["negro","amarillo"]
```

Ejecución desde Java con Jayway:

```
javac -cp "lib/*" -d bin src/jsonPath/Ejercicio1Inventario.java
java -cp "bin:lib/*" jsonPath.Ejercicio1Inventario
```

Por qué funciona

- En el inciso (a), `[*]` expande el arreglo `items` a todos sus elementos; sin el `[*]`, devolvería el array literal sin desempaquetar.
- En (b), la proyección `.nombre` aplicada a cada elemento del arreglo recolecta solo ese campo en un arreglo nuevo.
- En (c), el predicado `[?(...)]` compara la propiedad del nodo actual (`@.cantidad`) contra una referencia absoluta a la raíz (`$('stock_min')`). El estilo entre corchetes con comillas evita conflictos con el guión bajo o palabras reservadas.
- En (d), la conjunción de dos condiciones (`&&`) y la proyección final de `.colores[*]` achatan el array de arrays en un único arreglo de strings.

En MySQL 8+ (JSON)

MySQL 8+ soporta el tipo JSON nativo, los operadores `->` (extracción) y `->>` (extracción + unquote), y `JSON_TABLE` para “aplanar” arrays a filas relacionales.

```

-- Tabla con el documento como columna JSON
CREATE TABLE inventario_doc (
    id INT PRIMARY KEY,
    datos JSON NOT NULL
);

INSERT INTO inventario_doc (id, datos)
VALUES (1, '{"inventario":{"items":[...]},"stock_min":60}');

-- a) Todos los items del inventario
SELECT datos->>'$.inventario.items' AS items FROM inventario_doc;

-- b) Nombres de todos los items (JSON_TABLE aplana el array)
SELECT it.nombre
FROM inventario_doc d,
     JSON_TABLE(
         d.datos, '$.inventario.items[*]'
         COLUMNS (nombre VARCHAR(100) PATH '$.nombre')
     ) AS it;

-- c) Items bajos en stock (cantidad < stock_min)
SELECT it.nombre, it.cantidad
FROM inventario_doc d,
     JSON_TABLE(
         d.datos, '$.inventario.items[*]'
         COLUMNS (
             nombre VARCHAR(100) PATH '$.nombre',
             cantidad INT PATH '$.cantidad'
         )
     ) AS it
WHERE it.cantidad < JSON_VALUE(d.datos, '$.stock_min');

-- d) Colores de items con alto > 10 cm
SELECT it.nombre, it.colores
FROM inventario_doc d,
     JSON_TABLE(
         d.datos, '$.inventario.items[*]'
         COLUMNS (
             nombre VARCHAR(100) PATH '$.nombre',
             alto DECIMAL(10,2) PATH '$.tamanio.alto',
             unidad VARCHAR(20) PATH '$.tamanio.unidad',
             colores JSON PATH '$.colores'
         )
     ) AS it
WHERE it.unidad = 'cm' AND it.alto > 10;

```

Diferencias relevantes con JsonPath puro:

- JSON_TABLE es la herramienta clave: convierte un array JSON en filas SQL para poder usar WHERE, ORDER BY y joins habituales. Sin esto, hay que combinar varios JSON_EXTRACT con índices duros.
- Los operadores -> y ->> solo funcionan con *paths fijos* (no admiten predicados como [?(...)]); por eso para condiciones se necesita JSON_TABLE.
- Para indexar accesos JSON frecuentes en MySQL hay que crear *columnas generadas* (GENERATED ALWAYS AS (...) STORED) e indexar esas columnas; los índices funcionales sobre JSON_EXTRACT llegaron recién en 8.0.13.

3. Ejercicio 2: Generar un JSON único de facturas e items

Consigna

Realice un programa Java (ver biblioteca GSON) que cree un único documento json con la información almacenada en la tabla Factura y sus Items (base de datos del ejercicio 1) a de la práctica 1).

Idea

Se pide *aplanar* un modelo relacional (1:N entre `factura` e `item_factura`) a un único documento JSON con un arreglo de facturas y, dentro de cada factura, un arreglo anidado de items. Es el patrón canónico de exportación a un modelo documental; Gson se ocupa de la serialización segura (escape de comillas, codificación de números/fechas, manejo de `null`).

Solución

Esquema del documento generado:

```
{
  "generado_en": "2026-03-28T12:34:56.123-03:00",
  "facturas": [
    {
      "nro_factura": 1001,
      "nro_cliente": 1,
      "fecha": "2024-08-01",
      "monto": 106000.0,
      "items": [
        { "cod_producto": 102, "cantidad": 2, "precio": 8000.0 },
        { "cod_producto": 103, "cantidad": 1, "precio": 90000.0 }
      ]
    },
    ...
  ]
}
```

Estructura del proyecto Java:

- `recursos/json/src/jsonPath/Ejercicio2FacturaJson.java`: main del programa.
- `recursos/json/config/db.mysql.properties`: parámetros de conexión y nombres de tablas.
- `recursos/json/lib/gson-2.8.6.jar, mysql-connector-j-8.0.33.jar`: dependencias.

Flujo del programa:

1. lee la configuración del `.properties`;
2. abre una conexión JDBC a `practico1a_mysql`;
3. lee todas las facturas en un `Map<Long, Factura>`;
4. lee los items y los agrupa por `nro_factura`;
5. arma una raíz `{ generado_en, facturas[] }`;
6. serializa con `new GsonBuilder().setPrettyPrinting().create().toJson(raiz)`;
7. escribe el resultado en `salida/facturas_items.json`.

Bloque clave del programa (Gson):

```
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
```

```
Gson gson = new GsonBuilder()
    .setPrettyPrinting()
    .setDateFormat("yyyy-MM-dd")
    .create();

String json = gson.toJson(raiz);
Files.writeString(Path.of("salida/facturas_items.json"), json);
```

Compilación y ejecución:

```
javac -cp "lib/*" -d bin src/jsonPath/Ejercicio2FacturaJson.java
java -cp "bin:lib/*" jsonPath.Ejercicio2FacturaJson config/db.mysql.properties
```

Por qué funciona

- El paso 4 (agrupar items por `nro_factura`) es donde ocurre el “aplanado”: los items dejan de ser una tabla aparte y se anidan dentro de su factura.
- Gson maneja automáticamente el escape de strings (" dentro de un campo se convierte en \"), las fechas y los null; construir el JSON con `StringBuilder` es propenso a errores en cuanto los datos tienen caracteres especiales.
- `setPrettyPrinting()` indenta el JSON con dos espacios para que sea legible; en producción se suele omitir para ahorrar espacio.

En MySQL 8+ (JSON)

MySQL 8+ permite generar el mismo documento JSON *directamente en SQL*, sin pasar por Java. `JSON_OBJECT` arma un objeto, `JSON_ARRAYAGG` agrupa filas en un arreglo y se pueden anidar para construir documentos arbitrarios:

```
USE practico1a_mysql;

SELECT JSON_OBJECT(
    'generado_en', NOW(),
    'facturas', (
        SELECT JSON_ARRAYAGG(
            JSON_OBJECT(
                'nro_factura', f.nro_factura,
                'nro_cliente', f.nro_cliente,
                'fecha', f.fecha,
                'monto', f.monto,
                'items', (
                    SELECT JSON_ARRAYAGG(
                        JSON_OBJECT(
                            'cod_producto', i.cod_producto,
                            'cantidad', i.cantidad,
                            'precio', i.precio
                        )
                    )
                )
            )
        )
        FROM item_factura i
        WHERE i.nro_factura = f.nro_factura
    )
)
FROM factura f
) AS documento;
```

Diferencias relevantes con el enfoque Java+Gson:

- Una sola sentencia SQL reemplaza el programa Java entero: se ahorra todo el código de conexión, agrupado en memoria y serialización.
- `JSON_ARRAYAGG` no garantiza orden a menos que se use con una subconsulta `ORDER BY` (en MySQL 8.0.14+ se puede usar `... ORDER BY` dentro de la función agregada).
- Para volcar el resultado a archivo se usa `SELECT ... INTO OUTFILE` o se redirige la salida del cliente (`mysql -e "SELECT ..." > facturas_items.json`).

4. Ejercicio 3: Diseño de la base en MongoDB

Consigna

Cree una base de datos en MongoDB que contengan colecciones que permitan cargar datos con la siguiente estructura definida con un diagrama de E/R:

- **usuarios**: `nick`, `NyApe`, `edad`, `direccion` (compuesta por `calle` y `altura`), `telefonos`.
- **noticias**: `codigo`, `texto`, `fecha`; cada usuario *crea* 1:N noticias.
- **comentarios**: `codigo`, `texto`; cada usuario *hace* 1:N comentarios, y cada noticia tiene N comentarios *para* ella, con palabras clave.

Nota: El campo dirección está compuesto por calle y altura.

Idea

MongoDB es documental: el dato unitario es un documento JSON/BSON y se agrupa en *colecciones* (equivalente lógico de tablas pero sin esquema rígido). La diferencia clave con el modelo relacional es la decisión *embedding vs referencing*: atributos compuestos (`direccion`) y multi-valorados (`telefonos`) van **embebidos** dentro del usuario; entidades con ciclo de vida independiente (`noticias`, `comentarios`) van en colecciones **separadas** con referencias por clave de negocio.

Solución

Decisión de modelado:

- `direccion` y `telefonos` van dentro de `usuarios` (atributos del usuario, no entidades).
- `noticias` y `comentarios` se mantienen como colecciones aparte porque su volumen y ciclo de vida son independientes.

Esquema de las colecciones:

```
// usuarios
{
  nick:      "<clave de negocio, unica>",
  nyape:     "<nombre y apellido>",
  edad:      <int>,
  direccion: { calle: "<string>", altura: <int> },
  telefonos: [ "<tel1>", "<tel2>", ... ]
}

// noticias
{
  codigo:    <int, unico>,
  texto:     "<string>",
  fecha:     <Date>,
  autor_nick: "<ref logica a usuarios.nick>"
}
```



```

}

// comentarios
{
  codigo:          <int, unico>,
  texto:           "<string>",
  autor_nick:      "<ref logica a usuarios.nick>",
  noticia_codigo:  <int, ref logica a noticias.codigo>,
  pal_claves:      [ "<kw1>", "<kw2>", ... ]
}

```

Script de creación (mongosh) con *\$jsonSchema* e índices:

```

use("practico7_er_noticias");
db.dropDatabase();

db.createCollection("usuarios", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nick", "nyape", "edad", "direccion", "telefonos"],
    properties: {
      nick: { bsonType: "string", minLength: 1 },
      nyape: { bsonType: "string", minLength: 1 },
      edad: { bsonType: "int", minimum: 0 },
      direccion: { bsonType: "object",
        required: ["calle", "altura"],
        properties: {
          calle: { bsonType: "string", minLength: 1 },
          altura: { bsonType: "int", minimum: 0 }
        }
      },
      telefonos: { bsonType: "array", minItems: 1,
        items: { bsonType: "string", minLength: 1 }
      }
    }
  }
});

// ... similar para noticias y comentarios

db.usuarios.createIndex ({ nick:          1 }, { unique: true });
db.noticias.createIndex ({ codigo:         1 }, { unique: true });
db.comentarios.createIndex({ codigo:         1 }, { unique: true });
db.noticias.createIndex ({ autor_nick:     1 });
db.comentarios.createIndex({ autor_nick:     1 });
db.comentarios.createIndex({ noticia_codigo: 1 });

```

Por qué funciona

- **direccion y telefonos** embebidos: la consulta dominante es “traer un usuario completo”, así se evita un join lógico y la cardinalidad por usuario es chica (1 dirección, pocos teléfonos).
- **comentarios** en colección aparte (en vez de embebidos en la noticia): los comentarios crecen sin tope con la actividad, mantenerlos embebidos haría documentos cada vez más grandes; además tienen **codigo** propio y **pal_claves**, lo que los hace entidades de pleno derecho.
- **\$jsonSchema** aporta validación de tipos a la inserción, simulando los **CHECK** del modelo relacional sin perder la flexibilidad documental.
- Los índices únicos sobre **nick** y **codigo** aseguran la integridad de claves de negocio (MongoDB no tiene FKs, así que esta validación queda del lado de la aplicación).

En MySQL 8+ (JSON)

El mismo E/R se puede modelar en MySQL 8+ con columnas JSON. Para indexar atributos dentro del JSON (nick, autor_nick, apellido), se usan *columnas generadas STORED* sobre las cuales sí se puede crear un INDEX o FOREIGN KEY:

```
CREATE DATABASE practico7_er_mysql;
USE practico7_er_mysql;

CREATE TABLE usuarios (
  nick      VARCHAR(60) PRIMARY KEY,
  datos     JSON NOT NULL,
  apellido  VARCHAR(60) GENERATED ALWAYS AS (
    SUBSTRING_INDEX(JSON_UNQUOTE(datos->>'$.nyape'), ' ', -1)
  ) STORED,
  edad      INT GENERATED ALWAYS AS (
    CAST(JSON_EXTRACT(datos, '$.edad') AS UNSIGNED)
  ) STORED,
  INDEX idx_apellido (apellido),
  INDEX idx_edad     (edad)
);

CREATE TABLE noticias (
  codigo     INT PRIMARY KEY,
  datos      JSON NOT NULL,
  autor_nick VARCHAR(60) GENERATED ALWAYS AS (
    JSON_UNQUOTE(datos->>'$.autor_nick')
  ) STORED,
  fecha      DATE GENERATED ALWAYS AS (
    CAST(JSON_UNQUOTE(datos->>'$.fecha') AS DATE)
  ) STORED,
  INDEX idx_autor (autor_nick),
  INDEX idx_fecha (fecha),
  FOREIGN KEY (autor_nick) REFERENCES usuarios(nick)
);

CREATE TABLE comentarios (
  codigo      INT PRIMARY KEY,
  datos       JSON NOT NULL,
  autor_nick  VARCHAR(60) GENERATED ALWAYS AS (
    JSON_UNQUOTE(datos->>'$.autor_nick')
  ) STORED,
  noticia_codigo INT GENERATED ALWAYS AS (
    CAST(JSON_EXTRACT(datos, '$.noticia_codigo') AS UNSIGNED)
  ) STORED,
  INDEX idx_autor   (autor_nick),
  INDEX idx_noticia (noticia_codigo),
  FOREIGN KEY (autor_nick) REFERENCES usuarios(nick),
  FOREIGN KEY (noticia_codigo) REFERENCES noticias(codigo)
);
```

Limitaciones a notar:

- MySQL no tiene *pipeline* de agregación como MongoDB: las consultas analíticas se escriben con SQL estándar más funciones JSON.
- No hay \$lookup recursivo (\$graphLookup en MongoDB): los joins son los de SQL clásico.
- Los índices sobre JSON requieren columnas generadas; un INDEX directo sobre JSON_EXTRACT no se permite hasta crear primero la generated column.

5. Ejercicio 4: Carga mínima

Consigna

Agregue a la base de datos creada al menos dos usuarios, 4 noticias y dos comentarios por noticia.

Idea

Es una carga seed. La sutileza está en hacerla *idempotente*: los índices únicos creados en el Ejercicio 3 fallarían si se re-ejecutan los `insertOne`. El idiom de MongoDB es `updateOne(..., { upsert: true })`: si el documento existe se actualiza, si no se inserta.

Solución

Datos a insertar:

- 2 usuarios: moises, elias.
- 4 noticias con `codigo` ∈ {2001, 2002, 2003, 2004}.
- 8 comentarios (2 por noticia) con `codigo` ∈ {7001, ..., 7008}.

Patrón `upsert` para idempotencia:

```
db.usuarios.updateOne(
  { nick: "moises" },
  { $set: {
    nyape: "Moises Gonzalez",
    edad: NumberInt(45),
    direccion: { calle: "San Martin", altura: NumberInt(123) },
    telefonos: ["358-1111111", "358-2222222"]
  }},
  { upsert: true }
);

db.usuarios.updateOne(
  { nick: "elias" },
  { $set: {
    nyape: "Elias Perez",
    edad: NumberInt(38),
    direccion: { calle: "Belgrano", altura: NumberInt(450) },
    telefonos: ["358-3333333"]
  }},
  { upsert: true }
);

// 4 noticias (codigo 2001..2004), autor_nick alternando moises/elias
db.noticias.updateOne(
  { codigo: 2001 },
  { $set: {
    texto: "Nueva politica de privacidad",
    fecha: ISODate("2024-08-01"),
    autor_nick: "moises"
  }},
  { upsert: true }
);

// ... 2002, 2003, 2004 con autor_nick variable

// 8 comentarios (2 por noticia)
db.comentarios.updateOne(
  { codigo: 7001 },
  { $set: {
```

```

    texto: "Muy buena noticia",
    autor_nick: "elias",
    noticia_codigo: 2001,
    pal_claves: ["positivo", "actualidad"]
  }},
  { upsert: true }
);
// ... 7002..7008

```

Verificación de conteos:

```

db.usuarios.countDocuments({ nick: { $in: ["moises", "elias"] } }); // 2
db.noticias.countDocuments({ codigo: { $in: [2001, 2002, 2003, 2004] } }); // 4
db.comentarios.countDocuments({ noticia_codigo: { $in: [2001, 2002, 2003, 2004] } }); // 8

// Confirmar 2 comentarios por noticia
db.comentarios.aggregate([
  { $match: { noticia_codigo: { $in: [2001, 2002, 2003, 2004] } } },
  { $group: { _id: "$noticia_codigo", total: { $sum: 1 } } },
  { $sort: { _id: 1 } }
]);
// 4 filas, todas con total=2

```

Por qué funciona

- `updateOne` con `upsert: true` es el patrón estándar para cargas idempotentes en MongoDB: si la fila clave (`nick` para usuarios, `codigo` para noticias y comentarios) ya existe, se actualiza; si no, se inserta. Re-correr el script no rompe nada.
- `NumberInt(...)` fuerza el tipo `int` de BSON; sin esto, mongosh interpreta los números como `double`, lo que choca con el validador `$jsonSchema` que pide `bsonType: "int"`.
- `ISODate("...")` crea un valor `Date` de BSON; si se cargan strings la consulta del Ejercicio 5.c con `$sort` podría ordenar lexicográficamente en vez de cronológicamente.

En MySQL 8+ (JSON)

El equivalente MySQL usa `INSERT ... ON DUPLICATE KEY UPDATE` para mantener idempotencia. Los documentos JSON se escriben como literales de string:

```

USE practico7_er_mysql;

INSERT INTO usuarios (nick, datos) VALUES
('moises', JSON_OBJECT(
  'nyape',      'Moises Gonzalez',
  'edad',      45,
  'direccion', JSON_OBJECT('calle', 'San Martin', 'altura', 123),
  'telefonos', JSON_ARRAY('358-1111111', '358-2222222')
)),
('elias', JSON_OBJECT(
  'nyape',      'Elias Perez',
  'edad',      38,
  'direccion', JSON_OBJECT('calle', 'Belgrano', 'altura', 450),
  'telefonos', JSON_ARRAY('358-3333333')
))
ON DUPLICATE KEY UPDATE datos = VALUES(datos);

INSERT INTO noticias (codigo, datos) VALUES
(2001, JSON_OBJECT('texto', 'Nueva politica de privacidad',
  'fecha', '2024-08-01', 'autor_nick', 'moises')),
(2002, JSON_OBJECT('texto', 'Resultados del torneo',

```

```

        'fecha', '2024-08-05', 'autor_nick', 'elias')),
(2003, JSON_OBJECT('texto', 'Apertura del museo',
        'fecha', '2024-08-10', 'autor_nick', 'moises')),
(2004, JSON_OBJECT('texto', 'Cambio en el transporte',
        'fecha', '2024-08-15', 'autor_nick', 'elias'))
ON DUPLICATE KEY UPDATE datos = VALUES(datos);

-- comentarios (8 filas, 2 por noticia)
INSERT INTO comentarios (codigo, datos) VALUES
(7001, JSON_OBJECT('texto', 'Muy buena noticia', 'autor_nick', 'elias',
        'noticia_codigo', 2001, 'pal_claves', JSON_ARRAY('positivo', 'actualidad'))),
-- ... 7002..7008
ON DUPLICATE KEY UPDATE datos = VALUES(datos);

```

Diferencia clave con MongoDB: las columnas generadas (`apellido`, `autor_nick`, etc.) se calculan automáticamente con cada `INSERT/UPDATE`, así que no hay que duplicar la lógica de extracción al cargar datos. `ON DUPLICATE KEY UPDATE` reemplaza el documento completo cuando la PK ya existe (equivalente a `upsert`).

6. Ejercicio 5: Consultas MongoDB

Consigna

Resuelva las siguientes consultas:

- Listar el nick, nombre y apellido y edad de todos los usuarios.
- Listar los usuarios de más de 40 años.
- Devolver las noticias que crearon los usuarios de apellido Gonzalez ordenadas por fecha.

Idea

Tres consultas con dificultad creciente. La (a) requiere transformar un campo concatenado (`nyape`) en dos campos separados; el aggregation pipeline lo hace con `$split` y `$arrayElemAt`. La (b) es un `find` con filtro y proyección. La (c) requiere un *join lógico* entre `noticias` y `usuarios` (porque `autor_nick` es referencia, no embedding), que en MongoDB se resuelve con `$lookup` + `$unwind`.

Solución

a) *Listar nick, nombre, apellido, edad:*

```

db.usuarios.aggregate([
{
  $project: {
    _id: 0,
    nick: 1,
    edad: 1,
    nombre: { $arrayElemAt: [{ $split: ["$nyape", " "] }, 0] },
    apellido: { $arrayElemAt: [{ $split: ["$nyape", " "] }, -1] }
  }
}]

```

`$split` parte `nyape` por espacios; `$arrayElemAt` con índice 0 toma el primer token (nombre) y con -1 toma el último (apellido). La convención cubre nombres compuestos como "Juan Carlos Perez".

b) Usuarios de más de 40 años:

```
db.usuarios.find(
  { edad: { $gt: 40 } },
  { _id: 0, nick: 1, nyape: 1, edad: 1 }
);
```

c) Noticias de autores apellido Gonzalez ordenadas por fecha:

```
db.noticias.aggregate([
  {
    $lookup: {
      from: "usuarios",
      localField: "autor_nick",
      foreignField: "nick",
      as: "autor"
    }
  },
  { $unwind: "$autor" }, // $lookup devuelve array
  { $match: { "autor.nyape": /gonzalez$/i } }, // apellido termina en Gonzalez
  { $sort: { fecha: 1 } },
  {
    $project: {
      _id: 0,
      codigo: 1,
      texto: 1,
      fecha: 1,
      autor_nick: 1,
      autor: "$autor.nyape"
    }
  }
]);
```

Por qué funciona

- En (a), `$split + $arrayElemAt` reproduce la lógica de `SUBSTRING_INDEX` de MySQL pero dentro del pipeline. El `-1` en `$arrayElemAt` es el truco para tomar el último elemento sin conocer el largo.
- En (b) basta con `find`: el filtro `{ edad: { $gt: 40 } }` es directo y la proyección suprime `_id` (que MongoDB siempre incluye por defecto).
- En (c) el pipeline tiene cinco etapas: (1) `$lookup` simula el join por igualdad; (2) `$unwind` convierte el array de un solo elemento en un objeto plano; (3) `$match` filtra con regex insensible a mayúsculas; (4) `$sort` ordena cronológicamente (requiere que `fecha` sea `Date`, no `string`); (5) `$project` arma el resultado.
- Para que el `$lookup` sea eficiente con grandes volúmenes, conviene tener `usuarios.nick` indexado (lo cumple el índice único del Ejercicio 3).

En MySQL 8+ (JSON)

Las tres consultas se traducen a SQL con funciones JSON. Los índices sobre columnas generadas (apellido, autor_nick, fecha) hacen que los filtros y `ORDER BY` sean eficientes:

```
-- a) nick, nombre, apellido, edad
SELECT
  nick,
  SUBSTRING_INDEX(JSON_UNQUOTE(datos->>'$.nyape'), ' ', 1) AS nombre,
  SUBSTRING_INDEX(JSON_UNQUOTE(datos->>'$.nyape'), ' ', -1) AS apellido,
  JSON_EXTRACT(datos, '$.edad') AS edad
```

```

FROM usuarios;

-- b) Usuarios de mas de 40 años (usa el índice idx_edad sobre la columna generada)
SELECT
    nick,
    datos->>'$.nyape' AS nyape,
    edad
FROM usuarios
WHERE edad > 40;

-- c) Noticias de autores apellido Gonzalez ordenadas por fecha
SELECT
    n.codigo,
    n.datos->>'$.texto'      AS texto,
    n.fecha,
    n.autor_nick,
    u.datos->>'$.nyape'      AS autor
FROM noticias n
JOIN usuarios u ON u.nick = n.autor_nick
WHERE u.apellido = 'Gonzalez'
ORDER BY n.fecha;

```

Diferencias relevantes con MongoDB:

- `$lookup` + `$unwind` se reemplaza por un JOIN clásico. Es más simple y, con índices, más eficiente: el optimizador puede usar *hash join* o *merge join*.
- `$split` + `$arrayElemAt` se reemplaza por `SUBSTRING_INDEX(..., ' ', -1)`. Más conciso, pero menos expresivo si el formato del campo concatenado fuera más complejo.
- Para que la regex `/gonzalez$/i` de MongoDB sea equivalente, en MySQL se puede usar `REGEXP '(?i)gonzalez$'` en lugar de la igualdad estricta (`apellido = 'Gonzalez'`); la igualdad asume que la columna generada normalizó la capitalización.
- MySQL *no* tiene un equivalente directo del aggregation pipeline; los reportes analíticos se hacen con SQL + WITH (CTEs) o con GROUP BY clásico, y son menos componibles que la cadena de etapas de MongoDB.

Conclusiones

El práctico recorre tres capas del trabajo con datos semiestructurados:

1. **Consultas sobre JSON** (Ejercicio 1): JsonPath es a JSON lo que XPath es a XML. Ofrece operadores de navegación (`.`, `[*]`, `..`), filtrado (`[?(...)]`) y agregaciones.
2. **Generación de JSON desde código** (Ejercicio 2): el caso típico es leer un modelo relacional y emitir un documento JSON *aplanado*. Gson resuelve la serialización; la lógica de agrupado queda del lado de la aplicación. MySQL 8+ permite el mismo resultado en una sola sentencia SQL con `JSON_OBJECT` y `JSON_ARRAYAGG`.
3. **Modelado documental con MongoDB** (Ejercicios 3–5): las decisiones de embedding vs referencing, las claves de negocio y los índices únicos son las herramientas para representar un E/R sin perder integridad. El aggregation framework cubre la mayoría de las consultas no triviales, incluyendo joins lógicos con `$lookup`.

La idea común detrás del práctico es: *el modelo documental no es “relacional sin reglas”*. Tiene sus propias buenas prácticas (embedding por composición, referencing por relaciones independientes, validación con `$jsonSchema`, índices) y sus propias herramientas de consulta. Pero también es importante reconocer que **MySQL 8+ implementa JSON nativo** con funciones potentes: para muchos casos de uso donde no se necesitan replicación distribuida ni esquemas

verdaderamente dinámicos, una tabla con columna **JSON** y columnas generadas indexadas es una alternativa razonable a MongoDB, conservando transacciones ACID y SQL relacional.